

## Úloha C (Hammingov kód)

### Zadanie GNU Radio – Implementácia Hammingovho kódu (7,4) (Variant B, Úloha C)

#### Úvod a princíp riešenia

V tejto časti zadania som sa zamerlal na problematiku poprednej korekcie chýb (Forward Error Correction – FEC). Mojou úlohou bolo nahradiť dočasné prepojovacie bloky v schéme Variant\_B\_Hamming.grc plnohodnotným systémom Hammingovho kódu (7,4).

Tento kód je špecifický tým, že ku každým 4 informačným bitom pridáva 3 paritné bity, čím vzniká 7-bitové kódové slovo. Tento pomer (7,4) mi teoreticky umožňuje na strane prijímača identifikovať a úplne opraviť chybu jedného bitu v každom takomto bloku, čo je v digitálnej komunikácii kľúčové pre potlačenie vplyvu šumu.

#### Implementácia kódovania a dekódovania

Pre toto riešenie som vytvoril dva samostatné Python bloky typu `basic_block`. Výber tohto typu bloku bol nevyhnutný, pretože dochádza k zmene počtu vzoriek (v pomere 4:7 pri kódovaní a 7:4 pri dekódovaní), čo klasický synchrónny blok nezvláda.

- **Hamming Encoder** (Vysielač): Blok prijíma čisté bity. Pomocou maticových operácií a logických funkcií XOR generuje paritné bity  $p_1$ ,  $p_2$ ,  $p_3$ . Tieto bity sú v kóde presne usporiadané podľa štandardu Hamming (7,4) do výsledného prúdu, ktorý následne vstupuje do modulátora.
- **Hamming Decoder** (Prijímač): Toto je „mozog“ celého systému. Blok prijíma 7-bitové úseky a vypočítava z nich tzv. syndróm. Ak je syndróm nulový, dáta sú v poriadku. Ak je však nenulový, jeho hodnota mi presne ukazuje na pozíciu bitu, ktorý bol v kanáli poškodený šumom. Môj algoritmus tento bit automaticky invertuje (opraví) a na výstup pošle už len pôvodné 4 čisté bity.

#### Testovanie odolnosti a limitácie kódu

Funkčnosť systému som overoval pomocou nastaviteľného zdroja šumu (Noise Voltage). Sledoval som dva merače chybovosti: BER Raw (nechránený prenos) a BER Ham (prenos s mojou ochranou).

- Zistenie 1 (Úspešná oprava): Pri miernom zašumení kanála začal BER Raw okamžite stúpať. Môj dekóder však v reálnom čase všetky chyby opravoval, takže BER Ham zostával na nule. Toto je priamy dôkaz, že samoopravná schopnosť kódu funguje v praxi.
- Zistenie 2 (Limity kódu): Pri ďalšom zvyšovaní šumu som si všimol, že aj BER Ham začal pomaly rásť. Je to spôsobené tým, že Hamming (7,4) dokáže opraviť len jednu chybu na blok. Ak šum poškodí v jednom 7-bitovom slove dva alebo viac bitov, dekóder už nedokáže dáta korektne obnoviť.

Implementácia Hammingovho kódu preukázala výrazné zlepšenie kvality prenosu v zašumenom prostredí. Celý proces od kódovania cez prenos až po úspešnú opravu chýb je plne automatizovaný v rámci mojich Python blokov a integrovaný do výslednej schémy.

**Kód pre Encoder:**

```
import numpy as np
```

```
from gnuradio import gr
```

```
class blk(gr.basic_block):
```

```
    def __init__(self):
```

```
        gr.basic_block.__init__(self, name='Hamming (7,4) Encoder', in_sig=[np.uint8],  
out_sig=[np.uint8])
```

```
    def general_work(self, input_items, output_items):
```

```
        in0, out0 = input_items[0], output_items[0]
```

```
        n_blocks = min(len(in0) // 4, len(out0) // 7)
```

```
        if n_blocks == 0: return 0
```

```
        in_m = in0[:n_blocks*4].reshape((n_blocks, 4))
```

```
        d1, d2, d3, d4 = in_m[:,0], in_m[:,1], in_m[:,2], in_m[:,3]
```

```
        p1, p2, p3 = d1^d2^d4, d1^d3^d4, d2^d3^d4
```

```
        out_m = np.column_stack((p1, p2, d1, p3, d2, d3, d4))
```

```
        out0[:n_blocks*7] = out_m.flatten()
```

```
        self.consume(0, n_blocks * 4)
```

```
        return n_blocks * 7
```

**Kód pre Decoder:**

```
import numpy as np
```

```
from gnuradio import gr
```

```
class blk(gr.basic_block):
```

```
    def __init__(self):
```

```
        gr.basic_block.__init__(self, name='Hamming (7,4) Decoder', in_sig=[np.uint8],  
out_sig=[np.uint8])
```

```
    def general_work(self, input_items, output_items):
```

```
        in0, out0 = input_items[0], output_items[0]
```

```
        n_blocks = min(len(in0) // 7, len(out0) // 4)
```

```
        if n_blocks == 0: return 0
```

```
        in_m = in0[:n_blocks*7].copy().reshape((n_blocks, 7))
```

```
        p1, p2, d1, p3, d2, d3, d4 = [in_m[:,i] for i in range(7)]
```

```
        s1, s2, s3 = p1^d1^d2^d4, p2^d1^d3^d4, p3^d2^d3^d4
```

```
        syndrome = s1*1 + s2*2 + s3*4
```

```
        for i in range(n_blocks):
```

```
            if syndrome[i] > 0: in_m[i, syndrome[i]-1] ^= 1
```

```
        out_m = np.column_stack((in_m[:,2], in_m[:,4], in_m[:,5], in_m[:,6]))
```

```
        out0[:n_blocks*4] = out_m.flatten()
```

```
        self.consume(0, n_blocks * 7)
```

```
        return n_blocks * 4
```

## Obrázok výstupu:

MPSK channel with Noise and Hamming Encoding

— □ ×

BER MPSK

### BER Output without Hamming Encoding BER Output with Hamming Encoding

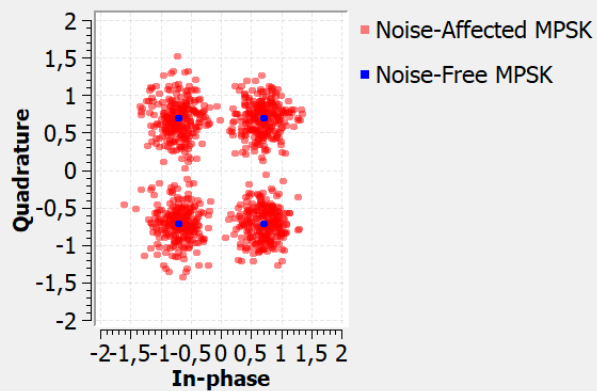
BER

BER

-4.474519 = X

-4.661040 = X

**Comparison of MPSK  
Signal  
Constellations:  
Noise-Free vs. Noise-  
Affected**



**MPSK Error Rate  
Histogram:  
Hamming Code vs.  
No ECC**

